

PvrGPU Shader 架構強化評估

Texture Cache 加 L2、以及 Coarse / Fine-Grained Shader Scheduling

專案：PvrGPU (Mesa Gallium PCO Driver + SystemC 功能模擬 C-Model)

範圍：src/systemc/ (texture、cache_mmu、shader) 與相關 driver 介面

日期：2026-09-04 撰寫依據：對 repo 快照的直接原始碼查證，非臆測

本文整理兩項針對 PvrGPU C-Model 的架構強化評估：（一）在現有 texture cache 路徑上加一層 L2，並讓「原路徑 / 新路徑」在模擬中可切換；（二）讓 shader 核心（USC）支援 coarse-grained 與 fine-grained 排程。兩項評估都以直接閱讀原始碼為依據，並標出各自涉及的檔案、可行性判斷與分階段實作建議。全文分析基於某一時間點的 repo 快照，另有其他 agent 正在同一 repo 上進行 GFXBench/Manhattan 相關修正，若程式碼在此之後又有變動，實作前應重新核對。

一、Texture Cache 加 L2 Cache

1.1 現況：cache 階層其實已經存在

Texture 取樣今天並不是直接讀記憶體的功能性路徑。USC 透過 sample_input / sample_output FIFO 把取樣要求交給 TPU (Texture Processing Unit, src/systemc/texture/texture_unit.cpp)，TPU 再經 cache_request / cache_response 把實際的 texel 讀取交給 TCU (Texture Cache Unit, src/systemc/cache_mmu/texture_cache.cpp)。TCU 是一顆 24KB、64B cache line、4-way write-back 的快取，有完整的 hit / miss / eviction 計數器與逐筆 WaitForCycles() 時序 (TextureCache::SampleRun, texture_cache.cpp 95-173 行)。TCU 後面接的是 SLC (System Level Cache, src/systemc/cache_mmu/slc.cpp, 2MB、128B、8-way)，且 SLC 已經有專門給 TPU 流量用的 texture_input / texture_output port。

也就是說，今天 TCU → SLC 這條路徑，在架構意義上已經接近「texture 專屬的 L1 快取 + 晶片共用的 L2 (SLC)」，跟 PowerVR 公開文件描述的 TPU 自帶快取加上 System Level Cache 是同一個形狀。所有這些 cache controller (TCU、SLC，以及尚未接上的 MixedCache、USC-L2) 都共用同一套可重用的 tag/data array 核心 src/systemc/cache_mmu/cache_array.h/.cpp (bank-interleave、write-back / write-allocate、true LRU)，所以要再加一層快取，並不是從零蓋一個新機制，而是沿用既有樣板。

1.2 現成、尚未接線的零件：USC-L2

src/systemc/cache_mmu/usc_l2_cache.* 已經用同一套 CacheArray 核心刻出一顆 8KB、64B、4-way 的快取，程式碼內部註明這是 scaffold，「USC load/store 的 response protocol 留給未來接上」。它目前綁定的是 USC 一般 load/store 流量，不是 texture，但形狀與命名慣例跟我們想加的 texture-L2 幾乎一致，可以直接照樣複製一份、換一組 config，而不是設計新的介面。

1.3 路徑可切換：已有前例可循

GpuMemorySystem 已經有三段式的 MemoryMode (kDirect / kBypass / kCache, gpu_memory_system.h 49-51 行)，TextureUnit 的建構子本身就是靠傳入不同物件來選路徑：傳 GpuMemorySystem* 走功能性直讀 / bypass / cache 路徑，傳 nullptr 並接上 cache_request / cache_response 就走 TCU 這條事件驅動的快取路徑 (texture_unit.cpp 1192-1229 行)。Driver 端則是 pvrgpu_cmd.c 讀取環境變數

PVRGPU_MODEL_MEMORY_MODE，轉成字串塞進 pvrgpu_systemc_api.h 的 memory_mode 欄位傳給 SystemC 端。這正是「原路徑 / 新路徑可切換」現成的機制先例——加一個新的 mode 字串（例如 cache_l2），理論上不會動到既有 kDirect / kCache 的行為。

1.4 待確認的方向

在動手前有一個設計選擇需要先定案：要在 TCU 與 SLC 之間再插一層「TCU (L1) → texture-L2 → SLC」三層架構，還是把現成但沒接的 usc_l2_cache 模組直接挪來給 texture 路徑重用？兩者實作量接近，但耦合方式不同，會影響是否要動到 SLC 既有的 texture_input / texture_output port。

1.5 預估影響檔案

檔案	變動內容
cache_mmu/texture_l2_cache.* (新增)	仿 texture_cache.* / usc_l2_cache.* 造一份，換用新的 config profile
cache_mmu/cache_array.h	視需要新增一組 config profile（容量 / line size / associativity）
texture/texture_unit.cpp (約 1177-1238 行)	fetch call site 加入可選路徑分支
cache_mmu/slc.h	視設計選擇，可能需要重接 texture_input / texture_output port
common/pipeline_state.h (CounterTxn)	新增 L2 的 hit/miss 計數器
gallium/drivers/pvrgpu/pvrgpu_cmd.c、pvr gpu_systemc_api.h	擴充 memory_mode 字串，加入新選項

結論：風險偏低、屬於附加式（additive）改動——現有的 TCU / SLC / USC-L2 / MixedCache 已經共用同一套模式，這比較接近「照樣板再刻一份加接線」，不是結構性改造。

二、Shader Coarse / Fine-Grained Scheduling

2.1 名詞界定

Coarse-grained multithreading (CGMT)：多個 thread/warp/instance

常駐，但平常一次只發射一個，只有遇到長延遲 stall（例如等 texture fetch）時才切到別的 context。Fine-grained multithreading (FGMT)：多個常駐 context，每個 cycle 都能輪替發射，用這個把記憶體延遲徹底藏起來——這也是真實 GPU（包括 PowerVR USC）維持高利用率的核心手段。

2.2 現況：完全序列化，沒有多工的前提

UscCluster::Run() (src/systemc/shader/usc_cluster.cpp 188-1346 行) 與 UscSlot::Run() (usc_slot.cpp 27-186 行) 都是單一個 SC_THREAD，一次只處理一個 PipelineTxn（一整批 vertex lane 或一整批 fragment quad group），處理完才讀下一個。同一批內的所有 lane 在 texture sample 的 suspend/resume 上是強制 lockstep——usc_cluster.cpp 505-513 行與 1134-1141 行會在 lane 之間對「是否還要再等一輪」意見不一致時直接丟例外（“lanes diverged across SMP rounds”）。遇到 texture cache 的回應，整個 cluster 是直接 blocking 在 texture_response_input->read() (429、1019 行)，沒有第二個 context 可以切換，連「等待時做點別的事」這個選項都不存在。

準確地說，現在的模型既不是 coarse 也不是 fine，而是完全序列化、單一個 in-flight batch——連最基本的多工前提（多個常駐 context）都還沒有。

2.3 與 texture cache 的依賴關係

這一點值得特別提醒：texture cache/L2 真正的價值，來自讓 miss latency 被「其他 context 的有用工作」蓋掉。如果排程仍是完全序列化、單一 context，不論 cache 做得多真實，對整體 throughput 的模擬意義都有限，只是把單一數字的延遲換一種算法，無法呈現「多執行緒藏延遲」這個 PowerVR 架構真正想展示的行為。因此若目標是讓整個模擬更貼近公開資料描述的行為，建議排程先於（或至少同時規劃）L2 cache 的細節。

2.4 分階段實作計畫

設計原則：每個階段都要附加式（additive）、向下相容——預設值或未啟用時，行為必須跟現在的模型完全一致，這樣才能安全地與其他 agent 目前在動的 pvr gpu_context.c 並行，不互相干擾。

Stage 0 骨架重構（不改變任何行為）

把 UscCluster::Run() 現在內嵌在單一 while 迴圈裡的處理邏輯，抽成一個可重入的「處理一個 context」函式，而不是寫死在迴圈本體。同時把 pco_iss.h 的

PcoVertexContinuation/PcoFragmentContinuation (263-281、313-323

行) 從「單一份」改成「陣列/表，index 為 context id」，但陣列大小先固定為 1，行為要求與今天 bit-identical。完成後應可直接用現有 regression 驗證零變化，作為之後每階段回頭比對的安全網。

Stage 1 texture request/response 協議加 tag

在 sample_input/sample_output (USC↔TPU) 與 cache_request/cache_response (TPU↔TCU) 的封包結構加入 context_id 欄位。這階段只加欄位、傳遞、驗證 (單一 context 時 tag 恆為 0) , 尚不做 reorder, 行為仍要求一致。涉及 pco_iss.h、texture_unit.h/.cpp 的 port 定義與 fetch call site (約 1177-1238 行) , 以及 cache_mmu/texture_cache.* 的 request/response struct。

Stage 2 常駐 context 表 + coarse-grained 的 switch-on-stall

核心改動。UscCluster::Run() 從「blocking read 一個 txn → 處理到完 → write output」改成維護一個固定大小 N (預設 config 給 N=2) 的 context slot 表：有空 slot 就從 input FIFO 收新 txn 進來；目前執行中的 context 遇到會 block 的讀取點時，標記為等待中並改執行其他已 ready 的 context。SystemC 上通常做成一個 dispatch thread 搭配每個 context 的狀態機，以 sc_event/notify() 驅動。新增 config 欄位 usc_resident_contexts (預設 1, 等於今天) 與 schedule mode, 放在 common/reference_uarch.h (22-39 行附近) 。

Stage 3 counters + regression 驗證

在 PipelineState::counters (對應 CounterTxn) 加入 context 相關統計：同時 occupied 的 context 數、切換次數、因切換省下的 idle cycle。先跑 N=1 確認與今天的 baseline (1849 pass/1 fail gl_manhattan) 完全一致，再針對 texture-heavy、multi-draw 案例驗證 N>1 時 cycle count 確實下降、且像素結果不變。可仿照 CMakeLists.txt 裡 indexed-geometry-test (568-591 行) 的樣板新增一個測試 target。

Stage 4 driver 端切換開關

比照 texture memory mode 的先例：pvrgpu_cmd.c 讀取新環境變數 (例如 PVRGPU_MODEL_USC_SCHEDULE_MODE=single|coarse) , 經 pvrgpu_systemc_api.h 傳給 SystemC 端，對應 Stage 2 的 config 欄位。

Stage 5 (暫緩) fine-grained 每 cycle round-robin

建議等 coarse-grained 版本的時序模型與 regression 都穩定後，再往上疊每 cycle 輪替的排程，風險較可控。

2.5 預估影響檔案

檔案	變動內容
shader/usc_cluster.cpp/.h	主戰場：context 表、issue 排程
shader/usc_slot.cpp/.h	admission 端需跟著調整
shader/pco_iss.h (及 pco_iss.cpp)	continuation 結構通用化為 per-context 表
texture/texture_unit.h/.cpp	sample_input/output 加 context tag
cache_mmu/texture_cache.*	request/response 帶 tag 原樣傳回
common/reference_uarch.h	新增常駐 context 數、排程模式 config
common/pipeline_state.h (CounterTxn)	新增排程相關計數器

gallium/drivers/pvrgpu/pvrgpu_cmd.c、pvrgpu_systemc_api.h	新增排程模式環境變數與傳遞
--	---------------

CMakeLists.txt	視需要新增排程驗證用 test target
----------------	------------------------

結論：這一項比 texture L2 明顯更大——不是可切換的 config，而是 UscCluster 核心迴圈的結構性改寫。不會動到 pvrgpu_context.c，與另一個 agent 目前的工作範圍沒有重疊。

三、建議順序與備註

建議先做 Stage 0（純骨架重構，行為不變、風險最低）——這階段不需要等 git/build 鎖解除即可開始撰寫，只是最終驗證 build 與 regression 時仍須等鎖清除。texture L2 的落地時機建議安排在 scheduling 的 Stage 2（常駐 context + coarse-grained）之後，這樣 L2 帶來的 miss latency 變異，才有排程可以真正把它藏起來，模擬數字才有意義。

本文所有檔案路徑、行號與現況描述，均為對某一時間點 repo 快照直接讀碼所得，非推測。實作前建議重新核對 live repo（尤其是另一 agent 仍在進行中的變動），以及本文引用行號在後續 commit 後是否已偏移。